

ISA Toolkit Refactor: Action Plan

Version 1.7 — Design Specification

Based on Smith-Miles & Muñoz (2023) and Simpson et al. (2025)

Prepared for review and incremental implementation

March 2026

Copyright © 2026 Mario Andrés Muñoz and contributors. Licensed under the PolyForm Non-Commercial License 1.0.0. See Section 11 for full terms.

Contents

1	Overview and Scope	3
1.1	Target MATLAB version	3
1.2	LIBSVM deprecation policy	3
1.3	Parameter passing: standalone functions vs. class	3
1.4	Unified training/test mode: eliminating PYTHIAtest and TRACetest	4
1.5	File disposition	5
2	Repository Hygiene (Pre-Phase 1 Prerequisite)	5
2.1	Files to delete	5
2.2	Files to retire at the start of Phase 7	6
2.3	Trial data relocation	6
2.4	Licence file replacement	6
3	Bug Fixes (Phase 1)	6
3.1	Critical crashes	6
3.1.1	exploreIS.m — typo causes unconditional crash	6
3.1.2	CLOISTER.m — de2bi requires Communications Toolbox	6
3.2	Algorithmic bugs	7
3.2.1	TRACE.m — custom dbscan shadows MATLAB native function	7
3.2.2	TRACE.m — division by zero in TRACEfitpoly	7
3.2.3	PRELIM.m — min(Y(:)) computed before NaN exclusion	7
3.2.4	PYTHIAtest.m — no handler for KNN or other native classifiers	7
3.3	Robustness fixes	7
3.3.1	PILOT.m — no rank-deficiency check before analytic branch	7
3.3.2	SIFTED.m (promoted from SIFTED2) — persistent cache replaces global	8
3.3.3	exploreIS.m — case-sensitive algorithm name matching	8
3.3.4	buildIS.m — addAttachedFiles for LIBSVM MEX left in parallel setup	8
3.3.5	exploreIS.m — calls PYTHIAtest and TRACetest directly	8
3.3.6	RNG policy	8
4	TRACE³: Unified Footprint Algorithm (Phase 3)	8
4.1	Design decision: single canonical implementation	8
4.2	Algorithm parameters	8

4.3	Algorithm steps	9
4.4	Stopping criteria	9
4.5	Integration with PYTHIA: mandatory coupling	9
4.6	Footprint metric naming	10
5	PILOT: 3D Extension, Viewpoint, and Alternatives (Phase 5)	10
5.1	3D projection	10
5.1.1	Analytic branch for 3D	10
5.2	Viewpoint optimisation	10
5.3	Alternative projection methods	11
5.4	Adjustable cost function weight	11
5.4.1	Note: topological preservation as a selection criterion, not an objective . .	11
5.5	SIFTED and dimensionality	12
6	PYTHIA: Generic Classifier Framework (Phase 4)	12
6.1	Classifier registry	12
6.2	Hyperparameter tuning	13
6.3	Unified prediction dispatch	13
6.4	Legacy model migration utility	13
7	InstanceSpace Class (Phase 7)	14
7.1	Class overview	14
7.2	Constructor	15
7.3	build() with stage control and option modification	15
7.4	explore() method	15
7.5	Backward-compatibility forwarding for buildIS and exploreIS	16
7.6	Test result access	16
7.7	save() and load()	16
8	Output and Visualisation (Phase 9)	17
9	Code Quality: Helpers and Namespace (Phases 2 & 10)	17
9.1	Named constants	17
9.2	Stage-level invocability	18
9.3	Package namespace	18
10	Reproducibility (Phase 7)	18
11	Licensing and Copyright	18
11.1	Chosen licence: PolyForm Non-Commercial 1.0.0	18
11.2	File header	18
12	Testing Strategy (Phase 1 and ongoing)	19
12.1	Reference dataset	19
12.2	Test scripts	20
13	Implementation Phases	20
13.1	Checkpoint criteria	26
14	Outstanding Questions and Deferred Items	26
A	Complete Options Reference	27

1 Overview and Scope

This document is the complete design specification for the refactor of the MATLAB Instance Space Analysis (ISA) toolkit. It supersedes Version 1.0 and incorporates all revisions from the review cycle of March 2026. It is grounded in two canonical references: Smith-Miles & Muñoz (2023) ACM CSUR and Simpson et al. (2025) ISA³ *Machine Learning*.

The refactor has five primary goals:

1. **Correctness.** Fix all identified bugs, including crashes, silent numerical errors, and incorrect algorithmic implementations relative to published pseudocode.
2. **Scientific fidelity.** Align the implementation precisely with the published algorithms, including TRACE³ (Algorithm 1 of Simpson et al.) and the 3D PILOT viewpoint optimisation (Equation 2 of Simpson et al.).
3. **Extensibility.** Replace hard-coded classifier (SVM/LIBSVM) and footprint logic with a configurable, strategy-based design supporting the full MATLAB `fitc*` family.
4. **Readability and maintainability.** Eliminate duplicated code, magic numbers, and fragile patterns; introduce named constants, extracted helpers, and a clean namespace.
5. **Class-based API.** Wrap the pipeline in a single `InstanceSpace` class with `build()` and `explore()` methods, stage-level execution control, and `save()/load()` support.

1.1 Target MATLAB version

R2025a or later. This permits use of native `dbscan`, `alphaShape` (2D and 3D), `fitcknn`, Bayesian hyperparameter optimisation via `fitc*` with `OptimizeHyperparameters`, and the `-v7.3` (HDF5-compatible) MAT file format.

1.2 LIBSVM deprecation policy

LIBSVM support is **deprecated as of this version**. New runs always use the MATLAB-native classifier registry. The LIBSVM dispatch is removed from `PYTHIA`. Users with existing models saved with LIBSVM classifiers should run the one-time migration utility:

```
ISAmigrateModel(rootdir)    % retrains classifiers using native registry; resaves
                             model.mat
```

The utility reads the existing `model.mat`, detects any LIBSVM struct, retrains using `opts.pythia.classifier` (default: `'knn'`), and overwrites the file.

1.3 Parameter passing: standalone functions vs. class

Each pipeline function (`PRELIM`, `SIFTED`, `PILOT`, `CLOISTER`, `PYTHIA`, `TRACE`) remains a standalone callable that accepts a plain `opts` struct. The `InstanceSpace` class does not change this: `build()` slices the relevant sub-struct from `obj.opts` and passes it to each function:

```
% Inside InstanceSpace.build() - class slices opts, functions stay standalone
obj.model.pythia = PYTHIA(obj.model.pilot.Z, obj.model.data.Ybin, ...
                          obj.model.data.Y,   obj.model.data.Ybest, ...
                          obj.model.data.algolabels, obj.opts.pythia);
```

This means the standalone functions have no dependency on the class and can be called independently for interactive inspection or custom workflows, preserving the existing usage pattern documented in the live demo. The class is purely an orchestrator; it does not own the algorithm logic.

1.4 Unified training/test mode: eliminating PYTHIAtest and TRACEtest

The separate PYTHIAtest.m and TRACEtest.m files are eliminated. Each pipeline function (PYTHIA, TRACE) operates in two modes distinguished by whether a fitted model sub-struct is supplied:

```
% Training mode: no prior model -> fits and returns a populated model
model.pythia = PYTHIA(Z, Ybin, opts);

% Test mode: prior model supplied -> predict only, no refitting
results.pythia = PYTHIA(Ztest, [], opts, model.pythia);
```

Internally, the function checks nargin and whether the fourth argument is non-empty. In training mode it fits classifiers and stores them in the output struct; in test mode it loads the stored classifiers and applies predict(). The same pattern applies to TRACE:

```
% Training mode
model.trace = TRACE(Z, Ybin, Yhat, spaceArea, opts);

% Test mode
results.trace = TRACE(Ztest, Ybin_test, Yhat_test, spaceArea, opts, model.trace)
;
```

This eliminates the code duplication between training and test paths, makes the InstanceSpace.explore() method trivial to implement (it simply calls each stage function in test mode), and ensures that future changes to a stage's logic propagate automatically to the test path.

1.5 File disposition

File	Disposition
buildIS.m	Becomes <code>InstanceSpace.build()</code>
exploreIS.m	Becomes <code>InstanceSpace.explore()</code>
PRELIM.m	Refactored in place; bugs fixed
SIFTED.m	Archived; SIFTED2.m promoted to replace it
SIFTED2.m	Promoted; persistent cache replaces global variable
PILOT.m	Extended: 3D, analytic 3D, PILOTviewpoint helper, PLS alternative, weight parameter
CLOISTER.m	Bug fixed (<code>de2bi</code> guard; pure-MATLAB replacement)
PYTHIA.m	Archived; PYTHIA2.m promoted to replace it
PYTHIA2.m	Promoted with classifier registry
PYTHIAtest.m	Retired. Merged into PYTHIA.m (test mode)
TRACE.m	Rewritten as single file; TRACE ³ canonical
TRACetest.m	Retired. Merged into TRACE.m (test mode)
FILTER.m	Dead variable removed
scriptcsv.m, scriptpng.m, scriptweb.m, scriptfcn.m scriptdisc.m	Preserved; scriptpng.m extended for 3D
example.m	Retired. Prints a console disclaimer only; redundant with LICENSE file and SPDX headers
older_scripts/	Deleted. Dead code (hardcoded private paths, outdated field names). Legacy option name mappings extracted to migration table (Section 6.4)
exampleWeb.m	Retired. Thin wrapper for <code>trainIS</code> (alias for <code>buildIS</code>); superseded by class API
testInterface.m	Retired. Thin wrapper for <code>testIS</code> (alias for <code>exploreIS</code>); superseded by <code>InstanceSpace.explore()</code>
trial/	Relocated to <code>test/data/</code> as the canonical reference dataset for the test suite (Section 12)
LICENSE	Replaced. GPL v3 text replaced with PolyForm Non-Commercial 1.0.0 (Section 11)
Contents.m	New. Required by MATLAB Central for version/date metadata (Section 13)
CITATION.cff	New. Machine-readable citation metadata (Section 13)

2 Repository Hygiene (Pre-Phase 1 Prerequisite)

These cleanup tasks must be completed *before* Phase 1 work begins, as they affect the file layout assumed by all subsequent phases.

2.1 Files to delete

The `older_scripts/` directory contains five files, all of which are dead code:

- `run_regression.m`, `run_timetable.m`, `run_bif1.m` — per-study driver scripts with absolute paths to a private OneDrive (`C:/Users/mariom1/...`). Useless to any other user. The *option field name mappings* they reveal (pre-refactor names for `opts.pilot`, `opts.pythia`, etc.) have

been extracted into the migration table (Section 6.4) and are the only actionable content.

- `unigoodreg.m` — entirely commented-out visualisation snippet. No recoverable logic.
- `example_maxflow2.m` — alternative `example.m` variant using LIBSVM (`uselibsvm=true`). Its only informational value is confirming that LIBSVM was the historical default, which is already documented.
- `instancesProjectionInterfaceWeb.m` — single-line wrapper calling `testIS(rootdir)`, where `testIS` is itself a deprecated alias for `exploreIS`. Redundant.

2.2 Files to retire at the start of Phase 7

`exampleWeb.m` and `testInterface.m` are MATILDA web-platform wrappers that call `trainIS` and `testIS` respectively. Since `trainIS` $\equiv$ `buildIS` and `testIS` $\equiv$ `exploreIS`, and both names are undocumented aliases, these files are retired when the `InstanceSpace` class replaces the script-based API. The `InstanceSpace` class provides the replacement entry points; no backward-compatibility forwarding is required for these internal wrappers.

2.3 Trial data relocation

The `trial/` directory contains `metadata.csv` and `metadata_test.csv`, the canonical reference dataset (Muñoz et al. 2018 classification study). These files are committed to the repository despite `*.csv` being listed in `.gitignore`; this is intentional. They are relocated to `test/data/` to co-locate them with the test suite, and the `.gitignore` exception is made explicit:

```
# .gitignore addition
!test/data/**
```

2.4 Licence file replacement

The current `LICENSE` file contains the GPL v3 text. This is replaced with the full PolyForm Non-Commercial 1.0.0 text as part of the licence change described in Section 11. This is a breaking change for any downstream fork that assumed GPL terms; the change should be announced in the release notes.

3 Bug Fixes (Phase 1)

These must be resolved before any new feature work, ordered by severity.

3.1 Critical crashes

3.1.1 `exploreIS.m` — typo causes unconditional crash

Location: Loop over `model.norm.lambdaX` entries.

Bug: `legnth()` is not a MATLAB function. The loop crashes whenever `opts.norm.normalizeX` is true.

```
for ii = 1:length(model.norm.lambdaX)    % was: legnth
```

3.1.2 `CLOISTER.m` — `de2bi` requires Communications Toolbox

Bug: `de2bi(0:2^nfeats-1, nfeats)` requires the Communications Toolbox, which may not be licensed.

Memory note: In practice, ISA selects at most 10–15 features for explainability (explaining 300 features is not useful; 10 is). At $q = 20$, the matrix V has $2^{20} \times 20 \approx 20M$ entries (~ 160 MB), which is acceptable. At $q = 10$ (typical), it is only $\sim 20,000$ rows. The guard is therefore a toolbox-availability fix rather than a memory fix.

Fix: Replace `de2bi` with a pure-MATLAB equivalent and retain the guard for safety:

```
MAX_FEATS = opts.cloister.maxFeatures;    % default: 20
if nfeats > MAX_FEATS
    warning('ISA:CLOISTER:tooManyFeatures', ...
        'CLOISTER skipped: %d features exceeds limit of %d.', nfeats, MAX_FEATS)
    ;
    out.Zedge = convhull(Z(:,1), Z(:,2));
    return;
end
% Pure-MATLAB replacement for de2bi (no toolbox required)
V = rem(floor((0:2^nfeats-1)' .* 2.^(-(nfeats-1:-1:0))), 2);
```

3.2 Algorithmic bugs

3.2.1 TRACE.m — custom dbscan shadows MATLAB native function

The local `dbscan` subfunction signature (`dbscan(x,k)`) conflicts with MATLAB R2019a+ native `dbscan(x,epsilon,minpts)`. Since TRACE³ does not use DBSCAN at all, the legacy subfunction is simply removed from the new canonical `TRACE.m`. It is retained only in the archived `TRACE_legacy.m`.

3.2.2 TRACE.m — division by zero in `TRACEfitpoly`

```
% Before: NaN when elements == 0, triangle silently kept
if opts.PI > (goodElements/elements)
% After
if elements == 0 || opts.PI > (goodElements/elements)
```

This bug affects only the legacy TRACE; the new TRACE³ does not use polygon fitting.

3.2.3 PRELIM.m — `min(Y(:))` computed before NaN exclusion

```
Yshift = nanmin(Y(:)) + 1;    % was: min(Y(:)) + 1
```

3.2.4 PYTHIAtest.m — no handler for KNN or other native classifiers

Replaced by a unified dispatch (see Section 6.3).

3.3 Robustness fixes

3.3.1 PILOT.m — no rank-deficiency check before analytic branch

```
if opts.pilot.analytic && rank(Ftilde) < size(Ftilde,1)
    warning('ISA:PILOT:rankDeficient', ...
        'Feature matrix rank-deficient; falling back to numerical solution.');
```

```
opts.pilot.analytic = false;
end
```

3.3.2 SIFTED.m (promoted from SIFTED2) — persistent cache replaces global

The GA fitness function cached evaluations in a `global containers.Map`, which is not thread-safe under `parfor`. Replaced with a `persistent` variable, cleared at the start of each SIFTED call via an internal `clearCache()` sub-call.

3.3.3 exploreIS.m — case-sensitive algorithm name matching

```
idx = find(strcmpi(strtrim(model.algoLabels), strtrim(testName)));
```

3.3.4 buildIS.m — addAttachedFiles for LIBSVM MEX left in parallel setup

The `parpool` setup block in `buildIS.m` unconditionally calls `addAttachedFiles(myPool, {'svmpredict.mexw64', 'svmtrain.mexw64'})` (and platform equivalents). Since LIBSVM is deprecated, these calls are removed. Leaving them causes a warning (missing files) for all users who do not have the MEX files, even when `opts.pythia.uselibsvm = false`.

3.3.5 exploreIS.m — calls PYTHIAtest and TRACEtest directly

`exploreIS.m` calls `PYTHIAtest` and `TRACEtest` directly rather than through the unified dispatch. These calls must be updated to the new unified signatures (training mode vs. test mode via the fourth argument) when the functions are merged in Phase 4. This is noted here so that Phase 4 does not omit the `exploreIS` path when testing the unified dispatch.

3.3.6 RNG policy

A single consistent policy is enforced throughout. At `build()` entry, the RNG is seeded from `opts.general.seed` (default: 42) using `rng(seed, 'twister')`. Individual BFGS trials and GA evaluations use trial-local seeds `seed + i`, ensuring bit-identical results regardless of parallel scheduling (see Section 10).

4 TRACE³: Unified Footprint Algorithm (Phase 3)

4.1 Design decision: single canonical implementation

TRACE³ (Algorithm 1, Simpson et al., 2025) is the **sole canonical footprint algorithm** in the refactored code, applicable to both 2D and 3D instance spaces. The legacy TRACE (DBSCAN + alpha-shape) is moved to `TRACE_legacy.m` and selectable only via `opts.trace.method = 'legacy'`.

When `opts.trace.method = 'legacy'`, `opts.trace.contra` defaults to `true`, since contra-diction removal is the natural complement of the legacy triangulation approach. Users may set `opts.trace.contra = false` to disable it explicitly.

4.2 Algorithm parameters

The three KNN parameters identified in Simpson et al. (Section 4.2) are exposed as `opts` fields with their paper-validated defaults. They are not treated as arbitrary magic numbers but as *published, experimentally calibrated defaults* whose sensitivity is documented:

Parameter	Default	Guidance (from Simpson et al.)
<code>opts.trace.nn</code>	50	Select based on dataset size. Low values ($k = 10$) overfit to outlier clusters; high values ($k = 200$) may miss small genuine regions. Low sensitivity for $k \in [30, 100]$.
<code>opts.trace.prior</code>	[0.6, 0.4]	[bad weight, good weight]. Tightening (e.g. [0.8, 0.2]) increases purity at cost of coverage. Low sensitivity overall.
<code>opts.trace.PI</code>	0.6	Minimum acceptable purity. Should be $\geq \text{opts.trace.prior}(2)$; otherwise the purity constraint has no effect.

4.3 Algorithm steps

1. Train KNN on (Z, y_{bin}) with `opts.trace.nn` neighbours and prior `opts.trace.prior`.
2. Predict $\hat{y} \leftarrow \text{predict}(\mathcal{M}_{KNN}, Z)$.
3. Retain $Z_u = \{z_i \mid \hat{y}_i = 1 \wedge y_i = 1\}$.
4. Build initial alpha-shape over Z_u at minimum enclosing radius.
5. Compute: area/volume, count, good count, density, purity.
6. Obtain alpha spectrum A_S .
7. Iterate while $\text{purity} < \text{opts.trace.PI}$ and $i \leq |A_S|$: shrink alpha, set `RegionThreshold` = $\Phi.\text{measure}/20$, recompute metrics, check stopping criteria.

4.4 Stopping criteria

The iteration terminates when any of the following hold:

- $\Phi.\text{purity} \geq \text{opts.trace.PI}$. (*success*)
- $|Z_u| \leq \text{opts.trace.minInstances}$ (default: 4; fewer than 5 instances cannot form a valid polygon or polyhedron).
- $\Phi.\text{measure} < \text{opts.trace.minAreaFrac} \times \text{spaceArea}$ (default: 1% of bounded space). The variable `spaceArea` is computed once as the convex hull area/volume of Z at the start of the TRACE call, then passed as an argument to `TRACEbuild3`.
- Alpha spectrum exhausted.

4.5 Integration with PYTHIA: mandatory coupling

TRACE³ uses PYTHIA's predicted labels $\hat{y}$ directly. There is no internal KNN re-trained within TRACE when PYTHIA has already trained a classifier for the same algorithm. Training separate models would be redundant and inconsistent.

The interface is:

```

footprint = TRACEbuild3(Z, Ybin_i, Yhat_i, spaceArea, opts.trace);
%   Ybin_i : true binary labels for algorithm i   (n x 1)
%   Yhat_i : PYTHIA predicted labels for algorithm i   (n x 1)
%   Zu = Z(Yhat_i==1 & Ybin_i==1, :)   per Algorithm 1 line 4

```

If `opts.pythia.skip = true`, TRACE falls back to $Z_u = \{z_i \mid y_i = 1\}$ (no prediction filtering) with a warning. The `opts.trace.usesim` flag is removed; the coupling is now unconditional.

4.6 Footprint metric naming

A uniform field name is used regardless of dimensionality:

Field	2D	3D
footprint.measure	area (units ²)	volume (units ³)
footprint.measureLabel	'Area'	'Volume'
footprint.density	count / area	count / volume
footprint.purity	good / count	good / count

Summary tables and CSVs use `measureLabel` for the column header. Internal code always accesses `footprint.measure`.

5 PILOT: 3D Extension, Viewpoint, and Alternatives (Phase 5)

5.1 3D projection

The extension follows Section 3.1 of Simpson et al. directly. Problem (D) is generalised by setting $Z \in \mathbb{R}^{n \times 3}$, $A_r \in \mathbb{R}^{3 \times q}$, $B_r \in \mathbb{R}^{q \times 3}$, $C_r \in \mathbb{R}^{a \times 3}$. The BFGS solver, multi-start strategy, and topological preservation selection criterion are unchanged.

5.1.1 Analytic branch for 3D

The 2D analytic solution uses the top 2 eigenvectors of $[\tilde{F}; Y][\tilde{F}; Y]^\top$. The 3D extension extracts the top 3:

```
d = opts.pilot.dims;    % 2 or 3
X = [Ftilde; Y];
[V, ~] = eigs(X * X', d);
Br      = V(1:q, :);
Cr      = V(q+1:end, :);
% Rank-checked pseudo-inverse:
Ar      = (V' * X * Ftilde') / (Ftilde * Ftilde');
Z       = (Ar * Ftilde)';
```

If $\text{rank}(\text{Ftilde}) < q$, the analytic branch is skipped (see Section 2.3.1).

5.2 Viewpoint optimisation

The viewpoint optimiser is implemented as `PILOTviewpoint(Z, Y, opts)` following Equation (2) of Simpson et al.:

$$\min_V \|Y - VZ^\top\|_F^2 + \lambda \langle v_1, v_2 \rangle \quad \text{s.t.} \quad V \in \mathbb{R}^{3 \times 2}, V^\top V \approx I,$$

with $\lambda = 0.2$ fixed (paper-calibrated; sensitivity is low for $\lambda < 0.5$, not exposed as a user parameter). The function returns the azimuth and elevation angles corresponding to the cross product $v_1 \times v_2$. These are stored in the model and used by `scriptpng` and the interactive `ISArecallView` helper.

When `opts.pilot.viewGroups` is provided (a cell array of algorithm index vectors), one viewpoint is computed per group. The format is:

```
opts.pilot.viewGroups = {[1 2 3], [4 5 6]}; % two groups of 3 algorithms each
% Default: empty -> one global viewpoint over all algorithms
```

`PILOTviewpoint` is implemented using the BFGS solver in the same manner as the main `PILOT` routine, as shown in the uploaded `PILOTANGLE.m` code. The refactored version cleans up this code: removes the inline `errorfcn` definition and places it as a proper named subfunction, removes the redundant row-normalisation applied twice to `A`, and makes the `lambda` parameter a named constant.

5.3 Alternative projection methods

Selectable via `opts.pilot.method`:

<code>'standard'</code>	Default. The BFGS/analytic <code>PILOT</code> method (Smith-Miles & Muñoz, 2023; previously called <code>PBLDR</code>). Produces A_r , B_r , C_r jointly.
<code>'pls'</code>	Partial Least Squares (<code>plsregress</code>). Maximises covariance between the projection and Y . The PLS weight matrix $W \in \mathbb{R}^{q \times d}$ gives $A_r = W^\top$; the X -loading matrix P gives $B_r = P$; and the Y -loading matrix Q gives $C_r = Q^\top$. All three matrices are naturally available from <code>plsregress</code> output, maintaining full pipeline consistency. Suitable when performance labels dominate the feature correlation structure.

MDA (`fitcdiscr`) is **excluded**: while discriminant axes can serve as A_r , the B_r and C_r matrices must be fitted by separate pseudo-inversion and regression, breaking the joint optimisation guarantee. The performance reconstruction $C_r Z \approx Y$ is then not optimised in the same objective, making the space less interpretable.

Non-linear methods (e.g. UMAP with a regression objective) are **deferred**: they would undermine the linearity guarantee that makes the instance space interpretable (*“algorithm X struggles where feature F is high”*). UMAP is noted as a future research direction but not included in this refactor.

Both supported methods apply to 2D and 3D via `opts.pilot.dims`.

5.4 Adjustable cost function weight

$$\min \|\tilde{F} - B_r Z\|_F^2 + \alpha \|Y - C_r Z\|_F^2$$

`opts.pilot.alpha` (default: 1.0) scales the performance reconstruction term. The current equal-weight behaviour is preserved by default; users may increase α to emphasise performance trends at the expense of feature trends.

5.4.1 Note: topological preservation as a selection criterion, not an objective

The best BFGS solution across all `opts.pilot.nTries` restarts is currently *selected* by the Pearson correlation $\rho(\mathbf{d}_F, \mathbf{d}_Z)$ between pairwise Euclidean distances in the feature space ($\mathbf{d}_F = \|\tilde{f}_i - \tilde{f}_j\|$, computed once on $\tilde{F}$ before optimisation) and pairwise Euclidean distances in the instance space ($\mathbf{d}_Z = \|z_i - z_j\|$, recomputed for each candidate solution $Z = XA^\top$). This is the topological preservation criterion of Yarrow et al. (2014), also used in Muñoz et al. (2018) and present in the current `PILOT.m` code.

An alternative design would incorporate $-\beta\rho$ directly into the BFGS cost function, turning topological preservation into a third objective alongside feature and performance reconstruction. However, this is a genuine optimisation—not a shortcut—because:

- $\mathbf{d}_F$ is fixed (a property of the input data, computed once from $\tilde{F}$ before any optimisation begins).
- $\mathbf{d}_Z$ changes with each update to A , since $Z = XA^\top$.

- $\rho(\mathbf{d}_F, \mathbf{d}_Z)$ must therefore be re-evaluated at each BFGS step. Computing all $\binom{n}{2}$ pairwise distances of Z costs $O(n^2)$ per gradient evaluation. For $n = 212$ this is $\sim 22\text{K}$ pairs—fast. For $n > 2000$ it becomes the bottleneck relative to the $O(nm)$ cost of the reconstruction terms.
- The analytic branch, which provides a closed-form optimal solution to the quadratic reconstruction objective, would be invalidated because ρ is not quadratic in Z .

The current selection-after-optimisation approach (compute ρ once per restart, choose the best A) therefore remains the design, as it preserves the analytic branch and adds no per-iteration cost.

The option `opts.pilot.topoWeight` (default: 0, disabled) is reserved for future experimental use. When non-zero, $-\beta\rho$ is added to the numerical cost function only (analytic branch skipped). This is not activated in the current refactor.

5.5 SIFTED and dimensionality

The canonical SIFTED (promoted from SIFTED2) calls PILOT internally — specifically the analytic branch with `ntries = 5` — for evaluating feature subsets. It does *not* use PCA. When `opts.pilot.dims = 3`, the internal PILOT call uses 3 components:

```
pilotOpts.analytic = true;
pilotOpts.dims     = opts.pilot.dims;    % 2 or 3
pilotOpts.ntries   = 5;
out = PILOT(X(:,idx), Y, featlabels(idx), pilotOpts);
```

This ensures feature selection is consistent with the chosen dimensionality of the final projection, without requiring a separate SIFTED variant.

6 PYTHIA: Generic Classifier Framework (Phase 4)

6.1 Classifier registry

The hard-coded SVM is replaced by a named registry validated at `build()` entry:

<code>opts.pythia.classifier</code>	MATLAB function	Notes
'knn' (<i>default</i>)	<code>fitcknn</code>	
'svm'	<code>fitcsvm</code>	
'tree'	<code>fitctree</code>	
'nb'	<code>fitcnb</code>	Naive Bayes
'linear'	<code>fitclinear</code>	Logistic/SVM on sparse data
'ensemble'	<code>fitcensemble</code>	Covers Random Forest, AdaBoost, etc. via <code>opts.pythia.ensembleMethod</code> (default: 'Bag')

`fitcecoc` is **excluded from the registry**: PYTHIA trains one *binary* classifier per algorithm (good vs. not-good), so multi-class ECOC machinery is never required. All PYTHIA classification tasks are binary by construction.

All algorithms in the portfolio use the same classifier for consistency. The name is resolved by `ISAGetClassifierFcn(name)` which returns the function handle and default hyperparameter ranges. All classifiers are called uniformly; prediction at test time uses `predict(obj, Znorm)`.

6.2 Hyperparameter tuning

Default: Sobol (`opts.pythia.tuning = 'sobol'`). A scrambled Sobol sequence (lower discrepancy than a Latin hypercube; generated via `sobolset` and `net`) provides better space-filling over the hyperparameter domain with fewer evaluations than Bayesian optimisation. MATLAB's `bayesopt` is theoretically superior but incurs significant overhead fitting a Gaussian Process surrogate on each iteration; for the typical ISA problem size this is unnecessarily slow.

The default number of evaluations is 20 for Sobol (versus 30 for Bayes); Sobol's better coverage means fewer iterations are needed for equivalent quality.

'sobol' (<i>default</i>)	Scrambled Sobol sequence, 20 evaluations, k -fold CV.
'bayes'	MATLAB <code>bayesopt</code> , 30 iterations, k -fold CV. Use when optimal tuning is required and runtime is not a constraint.
'none'	Uses pre-supplied <code>opts.pythia.params</code> directly; no tuning performed.

Pre-supplied parameters always override tuning: if `opts.pythia.params` is non-empty, `opts.pythia.tuning` is ignored.

6.3 Unified prediction dispatch

The LIBSVM struct dispatch is removed. All native MATLAB `Classification*` objects are handled uniformly:

```
% In PYTHIA (test mode): unified dispatch for all native classifiers
for ii = 1:numAlgos
    Yhat(:,ii) = predict(model.pythia.classifiers{ii}, Znorm);
end
```

6.4 Legacy model migration utility

`ISAmigrateModel(rootdir)` migrates any model produced by a previous version of the toolkit to the new `InstanceSpace` format. The utility handles the full range of legacy model structures that may exist in the wild:

Legacy pattern	Migration action
LIBSVM struct in <code>model.pythia</code>	Retrain using the new classifier registry at <code>opts.pythia.classifier</code> (default: 'knn').
<code>model.pythia.svm</code> (old field name)	Copy to <code>model.pythia.classifiers</code> .
<code>model.pilot.A</code> without B, C	Not expected in any production model: B and C are always assigned in the same code block as A. No action required; field presence is verified and a warning raised if absent.
<code>model.trace</code> in old triangulation format	Recompute footprints using TRACE ³ with <code>model.pythia.Yhat</code> (if present) or <code>model.data.Ybin</code> .
Missing <code>completedStages</code> field	Infer from which sub-structs are present; populate.
<code>opts.pbldr</code>	Rename to <code>opts.pilot</code> .
<code>opts.oracle</code>	Rename to <code>opts.pythia</code> .
<code>opts.sbound</code>	Rename to <code>opts.cloister</code> .
<code>opts.footprint</code>	Rename to <code>opts.trace</code> .
<code>opts.corr.flag</code> / <code>opts.corr.threshold</code>	Merged into <code>opts.sifted</code> ; map threshold to <code>opts.sifted.corrThreshold</code> .
<code>opts.clust.flag</code>	Merged into <code>opts.sifted</code> ; no direct equivalent; if false, set <code>opts.sifted.flag = false</code> .
<code>opts.perf.MaxMin</code>	Rename to <code>opts.perf.MaxPerf</code> .
<code>model.data.bestPerformace</code> (typo)	Rename to <code>model.data.Ybest</code> .

The utility detects the model version from the presence or absence of key fields, applies the appropriate transformations, and writes the migrated `model.mat` to the same `rootdir`. The original file is backed up as `model_legacy.mat`.

```
ISAmigrateModel(rootdir) % auto-detects version, applies all
fixes
ISAmigrateModel(rootdir, 'backupSuffix', '_v1') % custom backup suffix
```

7 InstanceSpace Class (Phase 7)

7.1 Class overview

The class is named `InstanceSpace` (not `InstanceSpaceAnalysis`), consistent with the established naming of the ISA framework and the MATILDA web tools.

```
classdef InstanceSpace
    properties (Access = public)
        rootdir      (1,:) char
        opts          (1,1) struct
        model         (1,1) struct % training results
        testDirs      (1,:) cell % paths of explored test sets
        testResults   (1,:) cell % one struct per test set (integer
                                indexed)
        completedStages (1,:) cell % which pipeline stages have been run
    end

    methods (Access = public)
        function obj = InstanceSpace(rootdir, opts)
        function obj = build(obj, varargin) % varargin: 'stages', {'prelim', '
                                sifted', ...}
        function obj = explore(obj, testRootDir)
```

```

function      save(obj)
function results = getResults(obj, idx)
function      plot(obj, varargin)
end

methods (Static)
    function obj = load(rootdir)
end
end

```

7.2 Constructor

```

obj = InstanceSpace(rootdir);           % uses default opts
obj = InstanceSpace(rootdir, opts);     % user-supplied opts (merged with
    defaults)

```

The constructor reads `metadata.csv`, validates `opts` via `ISAValidateOpts`, fills defaults, and sets `completedStages = {}`. No computation is performed.

7.3 build() with stage control and option modification

Options can be changed on the object at any time, before the first `build()` call or between staged calls. Because `InstanceSpace` is a value class, changes to `obj.opts` do not take effect until the relevant stage is (re-)run:

```

% ---- Initial setup ----
obj = InstanceSpace(rootdir);           % reads data, validates opts; no
    computation
obj.opts.pilot.dims = 3;                 % change any option before build
obj.opts.pythia.classifier = 'svm';

% ---- Run in stages ----
obj = obj.build('stages', {'prelim', 'sifted', 'pilot'});
% Inspect the projection
figure; scatter(obj.model.pilot.Z(:,1), obj.model.pilot.Z(:,2));

% ---- Adjust and re-run one stage ----
obj.opts.pilot.alpha = 2.0;              % increase performance weight
obj = obj.build('stages', {'pilot'});    % re-run PILOT only; sifted output reused

% ---- Continue pipeline ----
obj.opts.pythia.tuning = 'bayes';        % switch tuning strategy before PYTHIA
obj = obj.build('stages', {'cloister', 'pythia', 'trace'});

```

The `'stages'` argument accepts any subset of:

```
{'prelim', 'sifted', 'pilot', 'cloister', 'pythia', 'trace'}
```

in any order; the function enforces the correct execution order internally. Each stage stores its results in a named sub-struct of `obj.model` (`obj.model.prelim`, `obj.model.sifted`, etc.), so re-running a stage overwrites only that sub-struct. Prerequisites are checked before each stage runs; if PILOT is requested but `obj.model.sifted` is absent, an informative error is raised.

After each stage completes, its name is appended to `obj.completedStages`, providing a clear audit trail of what has been computed and with which options.

7.4 explore() method

```
obj = obj.explore(testRootDir);
```

Reads `metadata_test.csv` from `testRootDir`, applies training normalisation, projects via the stored A_r , runs `PYTHIA` (test mode) and `TRACE` (test mode) using the unified dispatch (Section 6.3), writes outputs to `testRootDir`, and appends to `obj.testResults` and `obj.testDirs`.

7.5 Backward-compatibility forwarding for `buildIS` and `exploreIS`

The standalone scripts `buildIS.m` and `exploreIS.m` are **retained as thin forwarding wrappers** after Phase 7 is complete. This preserves compatibility with the MATILDA web platform, which calls these entry points directly:

```
% buildIS.m (post-Phase-7 version)
function model = buildIS(rootdir)
    obj = InstanceSpace(rootdir);
    obj = obj.build();
    model = obj.model;
    model.opts = obj.opts;
end

% exploreIS.m (post-Phase-7 version)
function out = exploreIS(rootdir)
    % Requires model.mat to already exist in rootdir (written by buildIS)
    obj = InstanceSpace.load(rootdir);
    obj = obj.explore(rootdir);
    out = obj.testResults{end};
end
```

The `exampleWeb.m` and `testInterface.m` wrappers (which call the deprecated `trainIS` and `testIS` aliases) are **not** preserved; they are retired as described in Section 2.

Out-of-distribution warning: if more than 5% of test instances are clipped to the training feature bounds, a warning is issued advising the user that the test instances may be outside the training distribution and that retraining with a combined dataset should be considered.

7.6 Test result access

```
results = obj.getResults();           % training results (model struct)
results = obj.getResults(1);          % first explore() result
results = obj.getResults(2);          % second explore() result
obj.testDirs{1}                       % path of first test set
```

7.7 `save()` and `load()`

`save()` writes `model.mat` to `rootdir` using the `-v7.3` flag (HDF5-compatible MAT format). This single file:

- Contains all numeric arrays (`Z`, `Ybin`, `Yhat`, footprint vertices, etc.) readable from Python (`h5py`), R (`rhdf5`), Julia (`HDF5.jl`), and C (`libhdf5`).
- Contains MATLAB classifier objects for resuming computation.
- Contains the complete footprint objects (`model.trace.good{i}`, `model.trace.best{i}`, `model.trace.space`), including the `alphaShape` or `polyshape` geometry objects, the measure (area/volume), density, and purity for each algorithm. These are fully accessible after `load()` for downstream analysis, plotting, and containment queries (`inShape/isinterior`).

The existing CSV output files continue to be written and serve human-readable and web-infrastructure purposes. `scriptcsv.m`, `scriptpng.m`, `scriptweb.m`, and `scriptfcn.m` are all preserved. `scriptdisc.m` is **retired**: it printed a disclaimer/licence notice to the console, which

is superseded by the LICENSE file and per-file SPDX headers. A separate explicit `.h5` file is **not** produced; the v7.3 MAT file provides cross-language access, and the CSVs provide structured human-readable output.

`load()` is a static method that reconstructs an `InstanceSpace` object from `model.mat`:

```
obj = InstanceSpace.load(rootdir);
% Footprints are immediately available:
fp = obj.model.trace.good{3}; % footprint for algorithm 3
fp.measure                    % area (2D) or volume (3D)
fp.purity                    % purity
fp.shape                     % alphaShape or polyshape object
isinterior(fp.shape, Ztest)  % containment query for test points
```

8 Output and Visualisation (Phase 9)

`scriptpng` is preserved and extended. The 3D additions are:

- `scatter3/trisurf/alphaShape` calls for 3D scatter and volume footprints.
- A compass overlay (coloured axes indicating z_1, z_2, z_3 directions), consistent with Figure 5 of Simpson et al.
- PNGs saved at the viewing angles obtained from `PILOTviewpoint` (one per algorithm group in `opts.pilot.viewGroups`, or one global view).
- One `.fig` file per footprint for interactive rotation in MATLAB.
- `ISarecallView(fig, groupIdx)` snaps an open figure to the stored viewing angle for a given algorithm group.

For 3D CLOISTER, the convex hull boundary is computed in 3D. Its display requires projection through the viewpoint matrix V obtained from `PILOTviewpoint`. This is deferred to Phase 9 and noted as an open item.

9 Code Quality: Helpers and Namespace (Phases 2 & 10)

9.1 Named constants

All magic numbers are replaced. Since SIFTED, TRACE, and PYTHIA each become a single canonical file, there is no duplication to consolidate across multiple files — the helpers listed below are all introduced *within* their respective single canonical implementations:

Helper / constant	Replaces	Location
<code>ISAdefaults()</code>	Scattered <code>isfield</code> defaults	<code>ISAdefaults.m</code>
<code>ISAvalidateOpts(opts)</code>	Scattered validation guards	<code>ISAvalidateOpts.m</code>
<code>ISAsubsetData(data, idx)</code>	Repeated subset block in <code>buildIS</code>	<code>ISAsubsetData.m</code>
<code>ISacomputeFootprintStats(...)</code>	Stats code within new <code>TRACE.m</code>	subfunction
<code>ISAPYTHIASummary(...)</code>	Summary block within new <code>PYTHIA.m</code>	subfunction
<code>ISAgetCodeClassifierFcn(name)</code>	Classifier dispatch in <code>PYTHIA.m</code>	subfunction
<code>PILOTviewpoint(Z, Y, opts)</code>	Inline code from <code>PILOTANGLE.m</code>	<code>PILOTviewpoint.m</code>
<code>ISAmigrateModel(rootdir)</code>	(new) LIBSVM migration utility	<code>ISAmigrateModel.m</code>

9.2 Stage-level invocability

Each pipeline function (PRELIM, SIFTED, PILOT, CLOISTER, PYTHIA, TRACE) remains a standalone callable MATLAB function. The `InstanceSpace.build()` method orchestrates them; users may call individual functions directly for inspection or custom workflows. This preserves the existing interactive capability of the toolkit.

9.3 Package namespace

All files are placed under a `+ISA` package folder. The public entry point is the `InstanceSpace` class (accessed as `ISA.InstanceSpace` or added to path directly). Helper functions (prefixed `ISA*`) are also in the package. Pipeline functions remain as conventional MATLAB functions within the package for direct invocability.

10 Reproducibility (Phase 7)

Each random trial i in PILOT's BFGS multi-start loop and SIFTED's GA is seeded independently:

```
parfor i = 1:opts.pilot.nTries
    rng(opts.general.seed + i, 'twister');
    A0 = 2*rand(d,q) - 1;
    % ...
end
```

The *set* of trials is reproducible regardless of parallel scheduling, making the best solution selected from this set deterministic. TRACE³'s per-algorithm footprints are fully independent and parallelised with `parfor` without RNG interaction.

11 Licensing and Copyright

11.1 Chosen licence: PolyForm Non-Commercial 1.0.0

The toolkit is released under the **PolyForm Non-Commercial License 1.0.0** (<https://polyformproject.org/licenses/noncommercial/1.0.0/>). This licence permits use, modification, and redistribution for non-commercial research and educational purposes; prohibits commercial use without prior written permission; does not impose copyleft on derivative works; and provides explicit patent and liability protections. A `LICENSE` file at the repository root contains the full licence text.

Note on the current repository licence: the repository currently carries a GPL v3 `LICENSE` file. Replacing it with PolyForm Non-Commercial is an intentional and breaking change: GPL v3 permits commercial use; PolyForm Non-Commercial does not. The change is recorded in the release notes for v2.0.0 and applied as part of the repository hygiene pass described in Section 2.

11.2 File header

The following header is added to every source file. It uses the `LicenseRef-` SPDX prefix (the correct convention for licences not yet in the official SPDX registry), a direct URL, and a one-sentence human-readable summary:

```
% -----
% Instance Space Analysis (ISA) Toolkit
% Copyright (c) 2026 Mario Andres Munoz Acosta and contributors
% School of Computing and Information Systems
```

```

% The University of Melbourne, Australia
%
% SPDX-License-Identifier: LicenseRef-PolyForm-Noncommercial-1.0.0
% License: https://polyformproject.org/licenses/
%   noncommercial/1.0.0/
%
% You may use, modify, and redistribute this software for non-commercial
% research and educational purposes only. Commercial use requires prior
% written permission. See the LICENSE file for full terms.
%
% Reference:
%   Smith-Miles, K. & Munoz, M.A. (2023). Instance Space Analysis for
%   Algorithm Testing. ACM Computing Surveys, 55(12), Article 255.
%   https://doi.org/10.1145/3572895
%
%   Simpson, C., Munoz, M.A., Kandanaarachchi, S. & Campello, R.J.G.B.
%   (2025). ISA3: A 3-dimensional expansion of
%   Instance Space Analysis.
%   Machine Learning, 114, 240.
%   https://doi.org/10.1007/s10994-025-06871-5
% -----

```

12 Testing Strategy (Phase 1 and ongoing)

12.1 Reference dataset

The primary reference dataset is the classification study from Muñoz et al. (2018, *Machine Learning*, 107(1), 109–147), which has now been provided in full. It comprises:

- **metadata.csv**: 212 training instances $\times$ 10 features $\times$ 10 algorithms. Features are the 10 selected in the paper (Section 4): $H(\mathbf{X})_{\max}$, H_c , MC_X , DN_{ER} , $SD(\nu)$, F_3 , F_4 , L_2 , N_1 , N_4 . Algorithms: NB, LDA, QDA, CART, J48, KNN, L-SVM, poly-SVM, RBF-SVM, RF. Performance metric: error rate (lower is better).
- **metadata_tst.csv**: 23 test instances, same feature and algorithm columns.
- **coordinates.csv**: reference Z coordinates (z_1 , z_2) for all 212 training instances, against which PILOT output will be compared.
- **projection_matrix.csv**: reference A_r (rows labelled $Z_{\{1\}}$, $Z_{\{2\}}$); note the current output uses $Z_{\{1\}}$ labels which will be corrected to Z_1 , Z_2 in the refactor (the braces are a LaTeX artefact in the CSV, not meaningful).
- **svm_table.csv**: reference PYTHIA metrics (accuracy, precision, recall, hyperparameters) using the KNN classifier with the parameters shown.
- **footprint_performance.csv**: reference footprint areas, densities, and purities.

Important notes on reference values and reproducibility:

- The paper used all 235 instances as one dataset. The provided CSVs use a 212/23 train/test split introduced after the paper; numerical values will therefore differ from the paper tables. **The provided CSV files are the authoritative reference**, not the paper.
- The reference CSV files contain **SVM results** (the classifier in use at the time they were produced). The refactored code uses KNN by default. PYTHIA metrics (accuracy, precision, recall, hyperparameters) will not match the reference closely; they are provided only as a structural sanity check that the output format is correct.

- Feature selection is not performed on this dataset in the reference run (all 10 features are used). SIFTED may be tested with `opts.sifted.K = 5` or `6`, but results will deviate from reference coordinates and footprint metrics.
- PILOT is stochastic (30 BFGS restarts from random initialisations). The Z coordinates are *not* expected to match the reference to numerical precision—they may be reflected, rotated, or scaled differently. Regression tests for PILOT should use the topological preservation correlation $\rho(\mathbf{d}_F, \mathbf{d}_Z)$ as the comparison metric (verify it is ≥ 0.75 , consistent with the paper’s value of 0.8026), not coordinate comparison.
- Footprint areas, densities and purities are compared with loose tolerance ($\pm 5\%$ relative), since they depend on the Z coordinates. Structural properties (purity > 0.95 for most algorithms, area ordering) are more informative than exact values.

Each phase will be validated by:

1. Running the reference dataset through the refactored pipeline.
2. Comparing Z coordinates, footprint metrics (measure, density, purity), and PYTHIA accuracy against stored reference outputs.
3. Checking that all **warning** and **error** conditions (clipping, rank-deficiency, CLOISTER skip) trigger correctly on constructed edge-case inputs.

12.2 Test scripts

A set of `test_*.m` scripts will be provided alongside the code:

Script	Covers
<code>test_prelim.m</code>	NaN handling, Box-Cox, IQR bounding, nanmin fix
<code>test_sifted.m</code>	Feature selection reproducibility, 2D vs 3D PCA gating, cache clearing
<code>test_pilot.m</code>	2D/3D outputs, analytic vs numerical, rank-deficiency fallback
<code>test_cloister.m</code>	Pure-MATLAB <code>de2bi</code> equivalence, guard trigger
<code>test_pythia.m</code>	Classifier registry, Sobol tuning, pre-supplied params
<code>test_trace.m</code>	TRACE ³ stopping criteria, <code>spaceArea</code> argument, PYTHIA coupling
<code>test_integration.m</code>	Full pipeline on reference dataset; numerical regression

13 Implementation Phases

Each phase is complete when: the reference dataset runs without errors; numerical outputs match stored reference values within tolerance; any new feature has a test case in `test_phase*.m`.

Phase 1 — Bug Fixes

Resolve all critical crashes and algorithmic bugs identified in Section 3 before any restructuring. The toolkit must be functionally correct as a baseline.

Scope: Seven critical crash fixes + six algorithmic/robustness fixes (see Section 3). Set up test harness with reference dataset.

Vectorisation items included in this phase:

PRELIM.m

Vectorisation opportunity — Box-Cox and z-score loops: The inner loops over features and algorithms (lines 92–107) call `boxcox` and `zscore` per column. `zscore` already accepts a

matrix and operates column-wise; `boxcox` does not. The loops cannot be fully eliminated, but the per-column pattern is correct since each column has its own λ . No change required; however, preallocating `X` before the loop (currently done) is already optimal.

Logic issue — `min(Y(:))` before NaN exclusion (**Bug §2.2.3**): Already identified. `out.minY = min(Y(:))` on line 85 should be `nanmin(Y(:))`.

Logic issue — **relative performance and zero values**: Lines 23–26 set zeros to `eps` before computing relative performance. This is correct but fragile: if `Ybest` is zero for many instances (e.g., all algorithms fail completely), the relative performance matrix becomes ≈ 1 everywhere. A warning should be issued when more than 5% of `Ybest` values are zero.

Logic issue — **tie-breaking loop (lines 51–56)**: The loop over all instances to break ties in the best-algorithm selection is $O(n)$ with branching. This can be vectorised:

```
% Current: O(n) loop with randi inside
for i = 1:size(Y,1)
    if multipleBestAlgos(i)
        aux = aidx(bestAlgos(i,:));
        P(i) = aux(randi(length(aux),1));
    end
end
% Replacement: vectorised random tie-breaking
tieRows = find(multipleBestAlgos);
for i = tieRows' % typically very few rows
    aux = aidx(bestAlgos(i,:));
    P(i) = aux(randi(numel(aux)));
end
```

The outer loop is retained since the number of rows is typically small, but the branch on `multipleBestAlgos(i)` is avoided for the common case.

Phase 2 — Code Quality: Constants, Helpers, Console Output

Low-risk, high-readability pass. No algorithmic changes.

Scope: Extract magic numbers to named constants (see Section on Code Quality). Extract repeated code blocks to helper functions. Standardise console output throughout.

Console output policy:

- **Retained:** one line per major pipeline event (stage start/complete, key decisions, timing). Always shown regardless of `opts.general.verbose`.
- **Retained (verbose only):** per-algorithm training progress, silhouette values, hyperparameter results, projection matrix display. Shown when `opts.general.verbose = true` (default: `true`).
- **Removed:** ~ 30 decorative separator lines (`---...---`) that carry no information. One consistent separator per stage boundary only.
- **Standardised format:** `fprintf(' [STAGE] message\n', ...)` replaces mixed `disp/fprintf` usage. Timing reported as `'Completed in %.1f s.\n'`.

Phase 3 — TRACE³: Canonical Footprint Algorithm

Rewrite `TRACE.m` as the single canonical implementation covering both 2D and 3D. `TRACetest.m` is retired; test mode is handled by the unified mode switch.

Scope: TRACE³ algorithm (Section 4), metric naming, stopping criteria, PYTHIA coupling, legacy method switch.

Phase 4 — PYTHIA: Classifier Registry and Tuning

Replace the hardcoded SVM/KNN logic with a generic classifier registry. `PYTHIAtest.m` is retired; unified dispatch handles test mode.

Scope: Registry (knn, svm, tree, nb, linear, ensemble), Sobol tuning, migration utility `ISAmigrateModel`, unified `predict()` dispatch.

Vectorisation and logic items included in this phase:

PYTHIA2.m

Logic issue — `out.boxcosnt` **typo:** Field name is `boxcosnt` (transposed letters) throughout. Rename to `out.numNeighbours` and `out.distance` to match the CSV column names and remove the misspelling.

Vectorisation opportunity — **confusion matrix padding (lines 65–80):** The block that pads a 1×1 confusion matrix to 2×2 is a 9-branch nested conditional. This can be replaced with a single vectorised assignment:

```
aux = confusionmat(Ybin(:,i), out.Ysub(:,i), 'Order', [false true]);
% confusionmat with explicit 'Order' always returns 2x2; padding block removed.
```

Passing `'Order'`, `[false true]` forces the output to always be 2×2 regardless of which classes appear in the data, eliminating the conditional block entirely.

Logic issue — **per-algorithm `rng('default')` reset (lines 56–57):** Each algorithm resets the RNG to default before fitting. Under the new reproducibility policy, algorithm i uses `seed = opts.general.seed + i`.

Vectorisation opportunity — **PYTHIA summary table (lines 141–165):** The summary cell array is built with 13 separate `num2cell` calls and index assignments. This is readable but slow for large portfolios. A helper function `ISAPYTHIASummary(out, algolabels, Y, Ybin, Ybest)` will encapsulate this and build the table in one pass.

Phase 5 — PILOT: 3D, Viewpoint, and PLS Alternative

Extend PILOT to 3D, add the viewpoint optimisation helper, and add PLS as an alternative projection method. MDA is excluded (breaks joint optimisation).

Scope: 3D matrices and analytic 3D branch, `PILOTviewpoint` helper, `opts.pilot.method = 'pls'`, `opts.pilot.topoWeight` reserved parameter, SIFTED 3D gating.

Vectorisation and logic items included in this phase:

PILOT.m

Logic issue — **hardcoded row labels in `out.summary`:** Line 153 hardcodes `{'Z_{1}','Z_{2}'}` regardless of dimensionality. For 3D, this will label only 2 of 3 rows. Fix:

```
rowLabels = arrayfun(@(k) sprintf('Z%d', k), 1:d, 'UniformOutput', false);
out.summary(2:end, 1) = rowLabels;
```

Logic issue — RNG save/restore pattern: Lines 79–88 save and restore the RNG state around `rng('default')` to generate starting points. This pattern (`state = rng; rng('default'); ...; rng(state)`) is correct but creates a hidden dependency on the global RNG. Under the new reproducibility policy, the starting points are seeded from `opts.general.seed` using the trial-local seeding approach (Section 10).

Vectorisation opportunity — `nanmean(nanmean(...))` in error function: The 3D error function (line 18) calls `nanmean` three times (over rows, columns, depth). This is equivalent to `nanmean(A(:))` and avoids intermediate matrix allocation:

```
% Before (3D)
nanmean(nanmean(nanmean((...).^2, 1), 2), 3)
% After
mean((Xbar - reshape(...)*reshape(...)*Xbar(:,1:n)).^2, 'all', 'omitnan')
```

The 2D version has the same pattern. Using `mean(..., 'all')` (R2018b+) is cleaner and avoids the triple-mean computation.

Phase 6 — SIFTED: Promotion, Parallelism Fix, Vectorisation

Promote `SIFTED2.m` to `SIFTED.m`. Fix the nested parallelism bug and global variable issue. Apply vectorisation improvements.

Scope: Persistent cache replacing global `mymap`, inner `parfor` → `for`, trial-local seeding, correlation selection vectorisation.

Vectorisation and logic items included in this phase:

SIFTED2.m

Logic issue — global `mymap` not thread-safe: Already identified (Bug §2.3.2). The global `containers.Map` is replaced with a persistent variable cleared at entry.

Logic issue — nested `parfor` inside GA cost function (lines 144–147): `parfor` inside a function called from `ga` (which itself may be parallelised) creates nested parallelism. MATLAB's parallel pool does not support nested `parfor`; the inner loop silently degrades to serial. The inner `parfor` should be replaced by a plain `for` loop, relying on the outer GA parallelism if enabled.

Logic issue — `rng('default')` inside `costfcn` (line 142): Resetting the RNG to default inside the cost function (called hundreds of times by the GA) makes the KNN cross-validation deterministic per call but resets any user-set seed. Replace with the trial-local seeding approach.

Vectorisation opportunity — correlation selection loop (lines 58–60):

```
% Current: loop over nfeats rows
for ii=2:nfeats
    out.selvars(unique(row(ii, rho(ii,:))>=opts.rho))) = true;
end
% Vectorised: logical mask over the sorted rho matrix
mask = rho >= opts.rho;           % nfeats x nalgos logical
selected = unique(row(mask));     % flat indices of features meeting threshold
out.selvars(selected) = true;
```

This eliminates the loop entirely for the common case.

Phase 7 — InstanceSpace Class

Wrap the pipeline in the `InstanceSpace` value class. Add stage control, option modification between stages, RNG reproducibility, and save/load.

Scope: Class definition, `build()` with 'stages' argument, `completedStages` tracking, `save()/load()`, constructor. See Section 7.

Phase 8 — `explore()` and Migration Utility

Convert `exploreIS.m` into `InstanceSpace.explore()`. Implement `ISAmigrateModel` for full legacy model migration.

Scope: `explore()` method calling each stage in test mode, `ISAmigrateModel` handling all legacy patterns (see Section 6.4).

Phase 9 — Output Consolidation and `FILTER` Fix

Extend visualisation for 3D. Fix the `FILTER.m` scope bug. Consolidate all output scripts.

Scope: `scriptpng` 3D extension, 3D CLOISTER boundary display (deferred item), `.fig` export.

Vectorisation and logic items included in this phase:

`FILTER.m`

Critical logic bug — `model.data.unif` assigned to undefined `model`: Line 62 assigns to `model.data.unif`, but `model` is not a parameter or persistent variable in `FILTER`. This is a stray reference to the workspace variable `model` from `buildIS.m` and works only when `FILTER` is called from a script (not a function). In the refactored class-based design this will crash. Fix: return `unif` as part of the output struct.

Vectorisation opportunity — pairwise distance loop: The $O(n^2)$ nested loop (lines 11–54) computing `pdist2` for each pair is the primary bottleneck in `FILTER`. Replace with vectorised `pdist`:

```
% Precompute full distance matrix once
Dx = squareform(pdist(X));
Dy = squareform(pdist(Y));
Db = ... % nxn logical: all(Ybin(i,:) & Ybin(j,:)) for each pair
% Then process upper triangle without pdist2 inside the loop
```

The full distance matrix is $O(n^2)$ memory, acceptable for $n \leq 5000$. For larger datasets, a KD-tree (`knnsearch`) can restrict the search to near-neighbours only.

Phase 10 — Package Namespace, Licence Headers, README

Final reorganisation. All files moved into the `+ISA` package namespace. Licence headers added. README updated to reflect actual toolbox requirements.

Scope: `+ISA` namespace, `LicenseRef-PolyForm-Noncommercial-1.0.0` headers on all files, README toolbox list corrected (Communications and Financial Toolbox no longer required), `example.m` updated for new class API.

README.md and live demo observations

The README and live demo reveal several aspects of the current public API that the refactor must preserve or explicitly retire:

- The `opts.selvars.feats` and `opts.selvars.algos` options for selecting subsets of features and algorithms are documented and used. These must be preserved in the refactored `opts` schema.
- The `opts.selvars.smallscaleflag` / `smallscale` / `fileidxflag` / `fileidx` subsetting options are documented. These are preserved.
- The README still lists the **Communications Toolbox** and **Financial Toolbox** as requirements. After the `de2bi` fix, the Communications Toolbox is no longer required. The Financial Toolbox dependency should be verified (`boxcox` is in the Statistics and Machine Learning Toolbox; no Financial Toolbox functions appear in the source). The README will be updated to reflect the actual toolbox requirements post-refactor.
- The live demo confirms the interactive stage-by-stage usage pattern (`buildIS` then manual inspection then `exploreIS`) that motivates the stage-control design of `InstanceSpace.build()`.

MATLAB Central File Exchange workflow

The repository is already linked to MATLAB Central File Exchange entry 75170 -`instancespace`. File Exchange supports automated release synchronisation via GitHub: when a tagged GitHub release is created, File Exchange can pick it up automatically. The following deliverables are required for this workflow to function:

Contents.m A `Contents.m` file in the repository root (or `+ISA/` package folder) is how MATLAB Central reads the version and date. It must follow the standard MATLAB convention exactly:

```
% Instance Space Analysis Toolkit
% Version 2.0.0 (R2025a) 01-Jan-2026
%
% Standalone pipeline functions
% PRELIM      - Data preprocessing (outlier bounding, normalisation)
% SIFTED      - Feature selection (correlation + GA clustering)
% PILOT       - 2D/3D projection (PBLDR algorithm)
% CLOISTER    - Empirical bound estimation
% PYTHIA      - Algorithm selection (classifier training/prediction)
% TRACE       - Footprint construction (TRACE3 algorithm)
%
% Class interface
% InstanceSpace - Full pipeline wrapper class
%
% Utilities
% ISAmigrateModel - Migrate legacy model.mat to current format
% ISAvalidateOpts - Validate and fill defaults in opts struct
%
% Example
% example       - Demo script using the reference dataset
```

CITATION.cff A `CITATION.cff` file enables GitHub’s “Cite this repository” button and is consumed by Zenodo on each tagged release (the repository already has a Zenodo DOI). A minimal template:

```
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
```

```

title: "Instance Space Analysis: A toolkit for the assessment of algorithmic
      power"
version: 2.0.0
date-released: 2026-01-01
doi: 10.5281/zenodo.4484107
authors:
  - family-names: "Munoz Acosta"
    given-names: "Mario Andres"
    orcid: "https://orcid.org/0000-0000-0000-0000"
  - family-names: "Smith-Miles"
    given-names: "Kate"
license: LicenseRef-PolyForm-Noncommercial-1.0.0
repository-code: "https://github.com/andremun/InstanceSpace"

```

Tagging and release workflow File Exchange reads GitHub releases (not raw commits). The workflow is:

1. Update `Contents.m` version/date line and `CITATION.cff` version field.
2. Commit: `git commit -m "Release v2.0.0"`.
3. Tag: `git tag v2.0.0 && git push --tags`.
4. Create a GitHub release from the tag (via the GitHub UI or CLI).
5. File Exchange picks up the release automatically within 24 hours.

Scope note The `Contents.m` and `CITATION.cff` files are project-specific and created once as part of Phase 10. They are not part of the reusable refactor methodology; the pattern is documented here for completeness and to ensure it is not forgotten at release time.

13.1 Checkpoint criteria

Each phase is complete when:

- The reference dataset (Muñoz et al. 2018) runs to completion without errors.
- Z topological preservation $\rho(\mathbf{d}_F, \mathbf{d}_Z) \geq 0.75$.
- Footprint structural ordering matches reference within $\pm 5\%$ relative tolerance.
- PYTHIA accuracy is within 5% of the reference KNN results.
- Any new feature has a corresponding test case in `test_phase*.m`.

14 Outstanding Questions and Deferred Items

The following items from the design review are resolved or deferred:

Item	Status
✓ HDF5 format	Resolved: MAT v7.3 + CSVs; no separate .h5
✓ TRACE/PYTHIA coupling	Resolved: TRACE requires PYTHIA output
✓ LIBSVM deprecation	Resolved: migration utility; removed from PYTHIA
✓ PYTHIAtest/TRACetest	Resolved: unified into single file per function, mode-switched by supplied model argument
✓ Class name	Resolved: <code>InstanceSpace</code>
✓ MDA	Resolved: excluded (breaks joint optimisation); PLS retained
✓ Non-linear projection	Resolved: deferred as future research
✓ Sobol vs Bayes default	Resolved: Sobol default; Bayes optional
✓ SIFTED projection method	Resolved: uses PILOT (analytic, 5 tries)
✓ <code>trace.contra</code> default	Resolved: <code>true</code> when <code>method = 'legacy'</code>
✓ ECOC in registry	Resolved: excluded (all PYTHIA tasks are binary)
✓ Reference CSV files	Resolved: Muñoz et al. (2018) data provided
✓ Licence	Resolved: PolyForm Non-Commercial 1.0.0 (replacing GPL v3)
✓ Stage option modification	Resolved: opts changeable between any stages
✓ Topological weight in cost	Resolved: reserved as <code>opts.pilot.topoWeight</code> , disabled by default; future research
✓ Repository licence conflict	Resolved: GPL v3 LICENSE replaced with PolyForm Non-Commercial in pre-Phase 1 hygiene pass
✓ <code>older_scripts/</code> cleanup	Resolved: deleted; legacy field name mappings extracted to migration table (Section 6.4)
✓ <code>trainIS/testIS</code> aliases	Resolved: confirmed as undocumented aliases for <code>buildIS/exploreIS</code> ; both wrapper files retired
✓ Trial data location	Resolved: <code>trial/</code> relocated to <code>test/data/</code>
✓ MATILDA backward compat	Resolved: <code>buildIS.m</code> and <code>exploreIS.m</code> retained as thin forwarding wrappers after Phase 7
✓ LIBSVM <code>parpool</code> attachment	Resolved: <code>addAttachedFiles</code> calls removed from <code>buildIS.m</code> in Phase 1
○ 3D CLOISTER boundary display	Deferred to Phase 9
○ <code>viewGroups</code> concrete format	Needs worked example before Phase 5
○ Ensemble hyperparameter API	Verify <code>fitcensemble OptimizeHyperparameters</code> support in R2025a before Phase 4

A Complete Options Reference

Field	Default	Description
<code>general.seed</code>	42	Master RNG seed
<code>general.verbose</code>	<code>true</code>	Print progress to console
<code>general.parallel</code>	<code>true</code>	Use <code>parfor</code> where safe
<code>prelim.iqrMultiplier</code>	5	Feature bounding: median $\pm N \times$ IQR
<code>prelim.nanThreshold</code>	0.20	Max fraction of NaN values per feature
<code>prelim.bound</code>	<code>true</code>	Apply IQR bounding
<code>prelim.norm</code>	<code>true</code>	Apply Box-Cox + z-score

Field	Default	Description
<code>sifted.nClusters</code>	10	Number of k-means feature clusters
<code>sifted.corrThreshold</code>	0.3	Min Pearson correlation to retain feature
<code>sifted.maxFeatures</code>	10	Maximum features to select
<code>pilot.method</code>	<code>'standard'</code>	Projection method
<code>pilot.dims</code>	2	Projection dimensionality: 2 or 3
<code>pilot.nTries</code>	30	BFGS multi-start restarts
<code>pilot.analytic</code>	<code>true</code>	Use analytic solution if available
<code>pilot.alpha</code>	1.0	Performance term weight in cost function
<code>pilot.viewGroups</code>	<code>{}</code>	Algorithm groups for viewpoint optimisation
<code>cloister.maxFeatures</code>	20	Feature count guard before CLOISTER skips
<code>cloister.corrThreshold</code>	0.5	Correlation threshold for vertex pruning
<code>pythia.classifier</code>	<code>'knn'</code>	Classifier name from registry
<code>pythia.tuning</code>	<code>'sobol'</code>	Tuning strategy: <code>'sobol'</code> , <code>'bayes'</code> , <code>'none'</code>
<code>pythia.nTuningIter</code>	20	Number of Sobol/Bayes evaluations
<code>pythia.kFold</code>	5	Cross-validation folds
<code>pythia.params</code>	<code>{}</code>	Pre-supplied hyperparameters (overrides tuning)
<code>pythia.skip</code>	<code>false</code>	Skip PYTHIA entirely
<code>pythia.ensembleMethod</code>	<code>'Bag'</code>	Method for <code>fitcensemble</code>
<code>trace.method</code>	<code>'trace3'</code>	<code>'trace3'</code> or <code>'legacy'</code>
<code>trace.nn</code>	50	KNN neighbours (TRACE ³)
<code>trace.prior</code>	[0.6, 0.4]	KNN class prior: [bad, good]
<code>trace.PI</code>	0.6	Minimum purity threshold
<code>trace.minAreaFrac</code>	0.01	Min footprint fraction of space area
<code>trace.minInstances</code>	4	Min instances for a valid footprint
<code>trace.contra</code>	<code>false*</code>	Contradiction removal (* <code>true</code> if method= <code>legacy</code>)
<code>outputs.csv</code>	<code>true</code>	Write CSV files
<code>outputs.png</code>	<code>true</code>	Write PNG figures
<code>outputs.fig</code>	<code>true</code>	Write <code>.fig</code> files (3D mode)
<code>outputs.web</code>	<code>false</code>	Write web JSON output